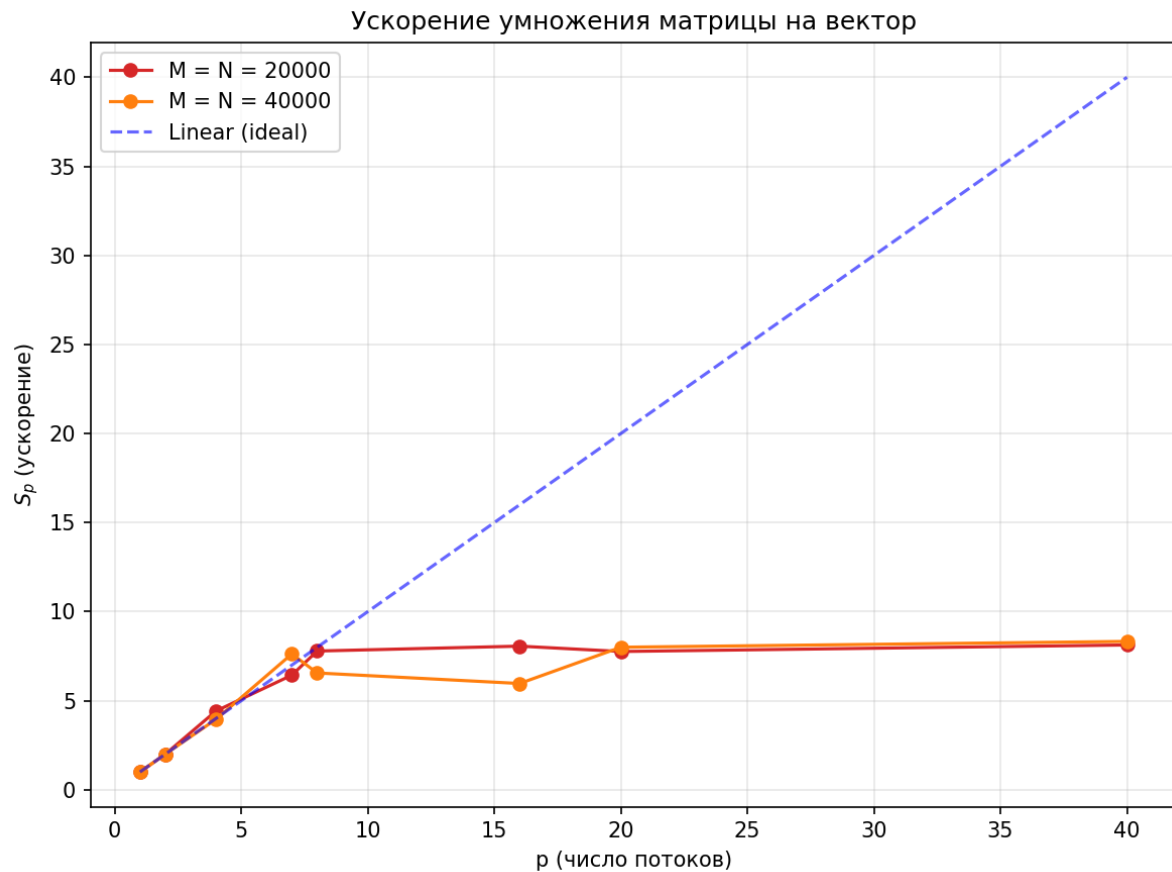


Лабораторная работа 3. Анализ эффективности

Задание 1. Многопоточное умножение матрицы на вектор (std::thread), double, параллельная инициализация массивов. Сервер: 80 ядер, 754 ГиБ ОЗУ.

График ускорения $S_p(p)$



M = N = 20000 (~3 ГиБ)

p	T _p , с	S _p = T ₁ /T _p	E _p = S _p /p
1	0.410854	1.00	1.00
2	0.205484	2.00	1.00
4	0.092943	4.42	1.11
7	0.063831	6.44	0.92
8	0.052746	7.79	0.97
16	0.050949	8.06	0.50
20	0.052916	7.76	0.39
40	0.050537	8.13	0.20

M = N = 40000 (~12 ГиБ)

p	T _p , с	S _p = T ₁ /T _p	E _p = S _p /p
1	1.610630	1.00	1.00
2	0.805389	2.00	1.00

4	0.407274	3.95	0.99
7	0.211192	7.63	1.09
8	0.245559	6.56	0.82
16	0.269903	5.97	0.37
20	0.201264	8.00	0.40
40	0.193353	8.33	0.21

Вывод о масштабируемости

Умножение матрицы на вектор ограничено пропускной способностью памяти (memory-bound): на каждый элемент приходится одно умножение и сложение, а матрица целиком читается из памяти. До 7-8 потоков ускорение растёт почти линейно (S_p близко к p , эффективность около 1), затем выходит на плато порядка 8х: потоки упираются в пропускную способность памяти. Бóльшая матрица (40000) использует ресурсы чуть лучше. Колебания в средней части объясняются конкуренцией за кэш и NUMA-эффектами.